

Министерство образования Новосибирской области
ГБПОУ НСО Новосибирский авиационный технический колледж имени Б.С.
Галущака

ОТЧЕТ
по практическому занятию 15
**Автоматизация сборки и деплоя проекта через GitHub Actions,
GitLab и Docker**
на основе веб-приложения «ТехноМастер»

Выполнил студент группы ПР-25.106
Ушаков Юрий Сергеевич

Новосибирск 2026

Цель и исходный проект

Цель работы - собрать веб-приложение в Docker, автоматизировать проверку кода и публикацию образа через GitHub Actions. Для работы использован существующий проект «ТехноМастер»: сервис ремонта техники с каталогом, заказами и личными кабинетами.

Приложение написано на Next.js и TypeScript. База данных и вход пользователей работают через Supabase. Проверки и контейнеризация добавлены к существующему репозиторию; страницы и пользовательские сценарии сохранены. Работа выполнена 6 октября 2026 года.

Таблица 1 - Среда выполнения

Среда	Использованное значение
Проект	your0kk/technomaster
Node.js в сборке	24
Next.js	16.3.6
Docker Desktop	4.94.0
Docker Engine	29.8.2, Linux amd64
Runner Actions	ubuntu-latest
Реестр образов	GitHub Container Registry

Практическая часть методички требует Docker и GitHub Actions. GitLab рассмотрен как аналог CI/CD: в нём конфигурация задаётся файлом .gitlab-ci.yml, а задания выполняет GitLab Runner. Отдельный конвейер GitLab в этой работе не создавался.

Схема автоматической сборки

Изменения сначала отправлены в отдельную ветку. Pull Request запускает проверки кода и контейнера. При успешном результате изменения объединяются с main. Событие push в main дополнительно разрешает публикацию образа в GHCR.

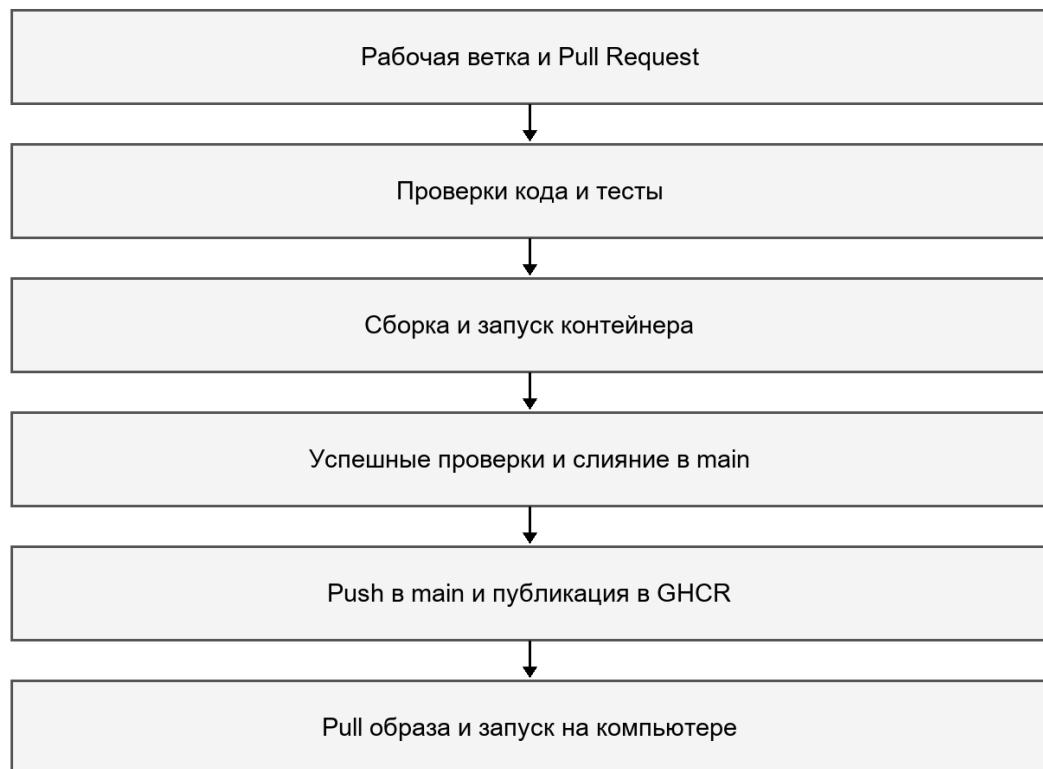


Рисунок 1 - Последовательность проверки и публикации

Публикуются два тега. latest указывает на последнюю сборку main. Полный хеш коммита позволяет выбрать конкретную версию независимо от последующих обновлений.

Dockerfile

Dockerfile состоит из трёх стадий. Сначала устанавливаются зависимости через `npm ci`, затем выполняется `production`-сборка. В итоговый образ копируются `standalone`-сервер Next.js, статические файлы и `public`. Инструменты разработки в итоговом образе не нужны.

```
# syntax=docker/dockerfile:1
FROM node:24-bookworm-slim AS dependencies
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM dependencies AS build
ARG NEXT_PUBLIC_SUPABASE_URL
ARG NEXT_PUBLIC_SUPABASE_ANON_KEY
ENV NEXT_TELEMETRY_DISABLED=1
COPY . .
RUN npm run build

FROM node:24-bookworm-slim AS runtime
LABEL org.opencontainers.image.source="https://github.com/your0kk/technomaster"
WORKDIR /app
ENV NODE_ENV=production \
    NEXT_TELEMETRY_DISABLED=1 \
    HOSTNAME=0.0.0.0 \
    PORT=3000
COPY --from=build --chown=node:node /app/.next/standalone ./
COPY --from=build --chown=node:node /app/.next/static ./next/static
COPY --from=build --chown=node:node /app/public ./public
USER node
EXPOSE 3000
HEALTHCHECK --interval=15s --timeout=5s --start-period=20s --retries=3 \
    CMD node -e
"fetch('http://127.0.0.1:3000/login').then(r=>process.exit(r.ok?0:1)).catch(()=>process.exit(1))"
CMD ["node", "server.js"]
```

Параметр `output` со значением `standalone` добавлен в `next.config.ts`. Контейнер запускается командой `node server.js`, слушает порт 3000 и работает от пользователя `node`. `HEALTHCHECK` проверяет доступность страницы входа.

Контекст сборки и настройки окружения

Файл `.dockerignore` исключает данные окружения, локальные пароли, кеш, Git, отчёты и файлы прототипа. Это уменьшает контекст сборки и не позволяет случайно включить закрытые файлы в Docker-образ. Ниже приведены основные правила; полный файл находится в репозитории.

```
node_modules
.next
.git
.github
.env*
*.local.json
*.pem
*.key
*.log
*.tsbuildinfo
coverage
test-results
.supabase
supabase
docs
output
tmp
assets
figma-plugin
```

`NEXT_PUBLIC_SUPABASE_URL`

и

`NEXT_PUBLIC_SUPABASE_ANON_KEY` нужны браузеру и задаются через `build arguments`. В GitHub они сохранены как `repository variables`. После изменения этих публичных настроек образ нужно пересобрать, потому что Next.js включает их в клиентский JavaScript.

`SUPABASE_SERVICE_ROLE_KEY` не является аргументом сборки и не записывается в `Dockerfile`. При локальном запуске он передаётся через `--env-file .env.local`. Этот файл хранится только на компьютере. Для входа workflow в GHCR используется временный `GITHUB_TOKEN` с разрешением `packages: write`.

Локальная сборка и запуск

В Docker Desktop запущен Linux Engine. Образ собран с именем `technomaster:practice-15`. Публичные настройки переданы сборщику из локального окружения, без серверного ключа. Затем контейнер запущен с файлом окружения и доступом только через локальный адрес.

```
docker build --build-arg NEXT_PUBLIC_SUPABASE_URL \
  --build-arg NEXT_PUBLIC_SUPABASE_ANON_KEY \
  -t technomaster:practice-15 .
docker run -d --name technomaster-practice-15 \
  --env-file .env.local -p 127.0.0.1:3005:3000 \
  technomaster:practice-15
docker ps
docker stop technomaster-practice-15
```

Команды приведены с переносами для чтения. Передача `build arguments` по имени предполагает, что публичные настройки уже заданы в окружении процесса. Для Windows команда может вводиться одной строкой. После проверки контейнер остановлен командой `docker stop`.

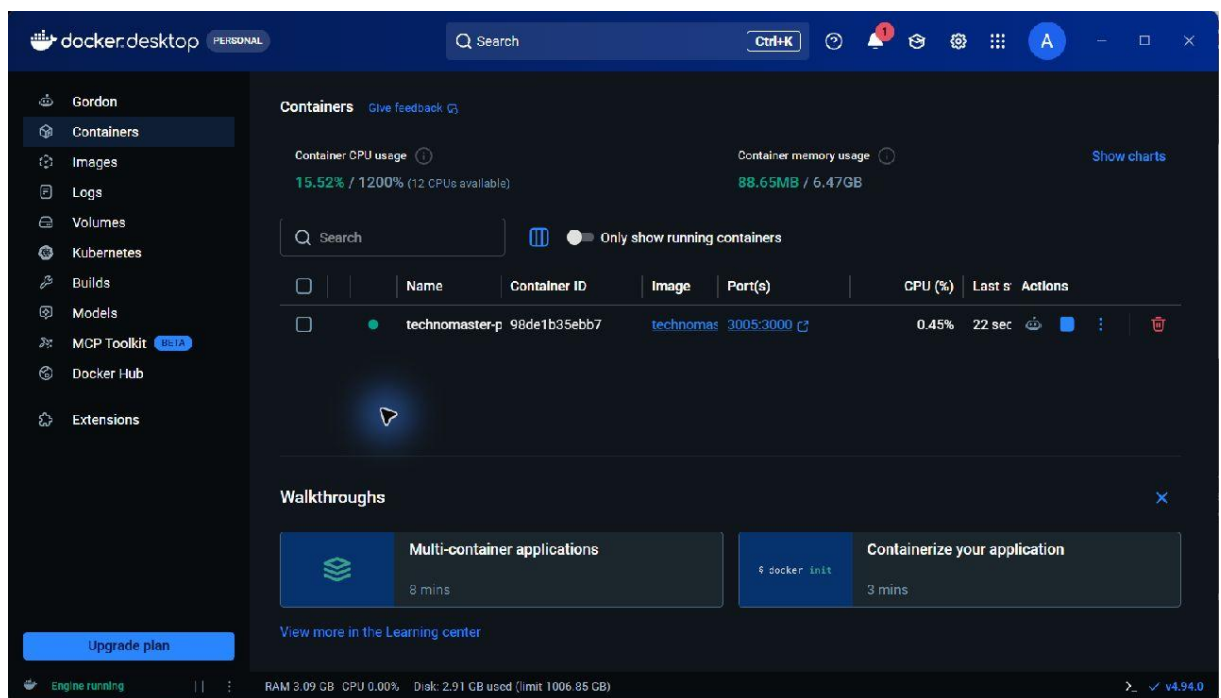


Рисунок 2 - Работающий контейнер в Docker Desktop

Проверка приложения в контейнере

Скрипт `scripts/check-container.mjs` запускает временный контейнер, ждёт готовности сервера и проверяет восемь страниц. Главная, услуги, каталог, корзина, оформление заказа, вход и кабинеты сотрудников ответили HTTP 200. Анонимный запрос к административному API получил HTTP 401.

Дополнительно проверены пользователь `node`, отсутствие серверного ключа в конфигурации образа и отсутствие `.env`, локальных файлов с реквизитами и приватных ключей внутри `/app`. В отдельном запуске с `.env.local` вход администратора и чтение заявок через контейнер дали HTTP 200. Данные базы при этой проверке не изменялись.

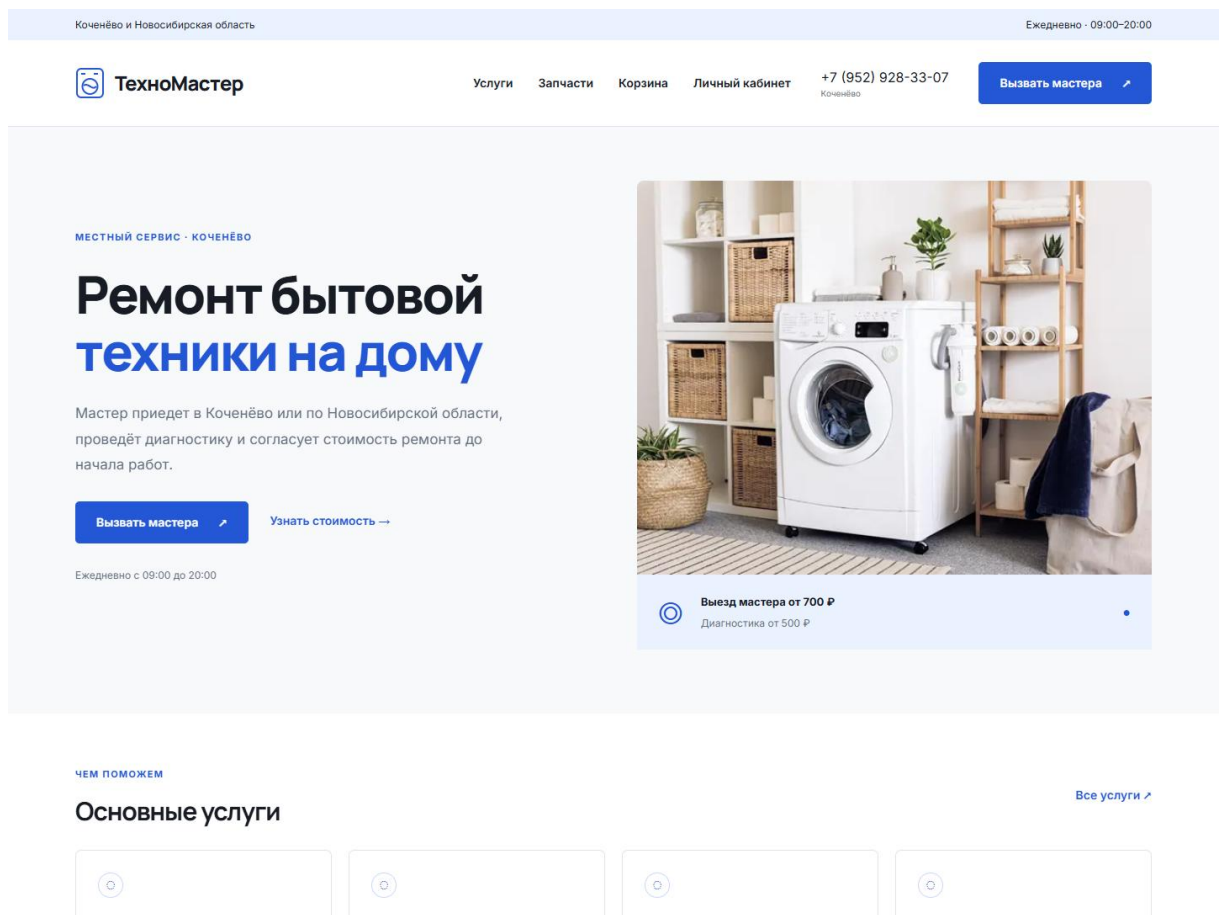


Рисунок 3 - Сайт из локального Docker-контейнера

GitHub Actions и проверка кода

Workflow находится в `.github/workflows/ci.yml`. Он запускается при `push`, `pull_request` и вручную. Задача `check` устанавливает Node.js, выполняет `npm ci`, линтер, проверку типов, пять тестов и `production`-сборку.

```
name: Проверка и сборка ТехноМастер
on:
  push:
  pull_request:
  workflow_dispatch:
permissions:
  contents: read
concurrency:
  group: ${ github.workflow }}-${ github.ref }}
  cancel-in-progress: ${ github.ref != 'refs/heads/main' }}
jobs:
  check:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 24
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test
      - run: npm run build
```

Права по умолчанию ограничены чтением репозитория. `needs: check` не позволяет запустить сборку контейнера при ошибке проверки кода. Для рабочих веток устаревшие запуски отменяются, чтобы не тратить runner на предыдущий коммит.

Сборка и публикация образа

Задача `container` использует `Docker Buildx`, собирает образ с публичными настройками и загружает его в локальный `Docker runner`. Перед публикацией запускается скрипт проверки контейнера. Кеш `GitHub Actions` ускоряет повторную сборку.

```
container:
  needs: check
  runs-on: ubuntu-latest
  timeout-minutes: 20
  permissions:
    contents: read
    packages: write
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 24
    - uses: docker/setup-buildx-action@v3
    - name: Сборка образа
      uses: docker/build-push-action@v6
      with:
        context: .
        load: true
        tags: |
          ghcr.io/your0kk/technomaster:latest
          ghcr.io/your0kk/technomaster:${{ github.sha }}
        build-args: |
          NEXT_PUBLIC_SUPABASE_URL=${{ vars.NEXT_PUBLIC_SUPABASE_URL }}
          NEXT_PUBLIC_SUPABASE_ANON_KEY=${{ vars.NEXT_PUBLIC_SUPABASE_ANON_KEY }}
        cache-from: type=gha
        cache-to: type=gha,mode=max
    - name: Проверка контейнера
      run: node scripts/check-container.mjs ghcr.io/your0kk/technomaster:latest
    - name: Вход в GHCR
      if: github.event_name == 'push' && github.ref == 'refs/heads/main'
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
    - name: Публикация образа
      if: github.event_name == 'push' && github.ref == 'refs/heads/main'
      run: |
        docker push ghcr.io/your0kk/technomaster:latest
        docker push ghcr.io/your0kk/technomaster:${{ github.sha }}
```

Вход в GHCR и `docker push` выполняются только для `push` в `main`. `Pull Request` проверяет образ без публикации. Благодаря этому пакет в реестре соответствует проверенному состоянию основной ветки.

Работа с Pull Request

Для изменений создана ветка `codex/practice-15-docker`. После коммита и `push` открыт Pull Request №1 с целевой веткой `main`. В PR показаны изменённые файлы, описание контейнеризации и результаты проверок.

```
git switch -c codex/practice-15-docker
git add Dockerfile .dockerignore next.config.ts \
  .github/workflows/ci.yml scripts/check-container.mjs
git commit -m "Добавить Docker и публикацию образа"
git push -u origin codex/practice-15-docker
```

Обе задачи `pull_request` завершились успешно. После этого PR объединён с `main`. Такой порядок позволяет проверить изменения до их попадания в основную ветку и сохранить обсуждение, историю коммитов и результаты CI.

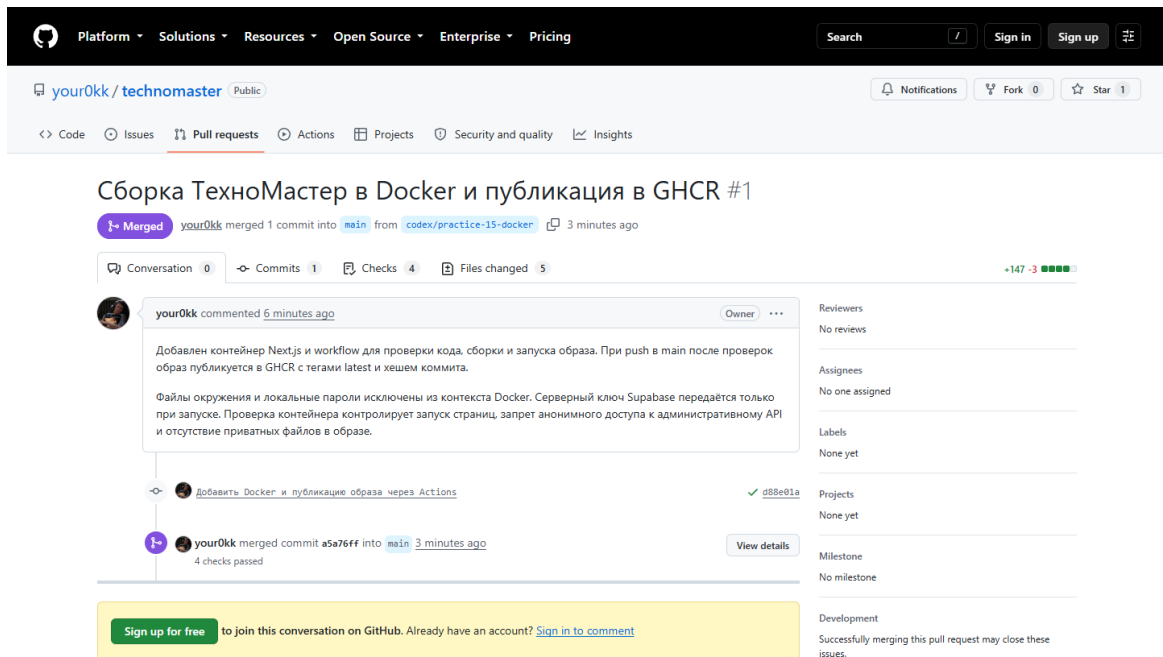


Рисунок 4 - Pull Request №1 в репозитории

Ссылка: <https://github.com/your0kk/technomaster/pull/1>

Результат GitHub Actions

Проверка PR и запуск после слияния завершились успешно. В запуске main выполнены обе задачи, включая авторизацию в GHCR и публикацию обоих тегов. Логи позволяют увидеть команду или шаг, на котором произошла бы ошибка.

The screenshot displays the GitHub Actions interface for the repository `your0kk/technomaster`. The workflow, titled "Добавить сборку Docker и публикацию в GHCR #15", is shown as successful. The workflow file is `ci.yml`, triggered on push. The workflow consists of two steps: `check` (51s) and `container` (1m 50s). The `Annotations` section shows two warnings: `check` and `container`, both related to deprecated Node.js 20 actions. The `check` step warning states: "Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-node@...". The `container` step warning states: "Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-node@...". Both warnings include a "Show more" link. The `check` step warning also includes a note: "The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/iss...". The `container` step warning also includes a note: "The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/iss...".

Рисунок 5 - Успешный workflow после слияния

Запуск PR: <https://github.com/your0kk/technomaster/actions/runs/37422189901>

Запуск main: <https://github.com/your0kk/technomaster/actions/runs/37422431162>

Образ в GitHub Container Registry

Опубликован пакет `ghcr.io/your0kk/technomaster`. Для проверки образ получен из реестра, запущен на компьютере и проверен тем же скриптом. Проверка опубликованного образа завершилась успешно. Серверные настройки передаются при запуске и не входят в пакет.

```
docker pull ghcr.io/your0kk/technomaster:latest
docker run -d --name technomaster-ghcr \
  --env-file .env.local -p 127.0.0.1:3006:3000 \
  ghcr.io/your0kk/technomaster:latest
docker ps
docker stop technomaster-ghcr
```

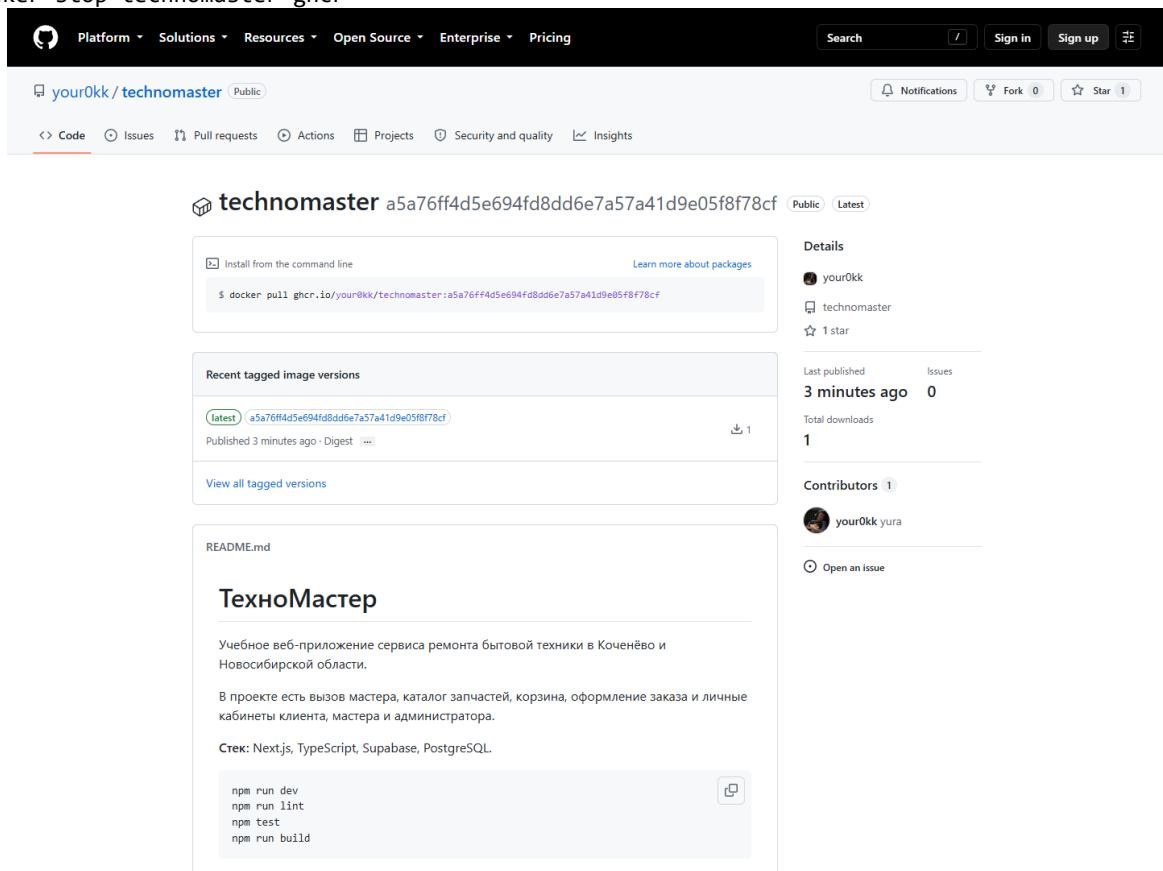


Рисунок 6 - Пакет и версии образа в GHCR

Версия по коммиту: `a5a76ff4d5e694fd8dd6e7a57a41d9e05f8f78cf`

Страница пакета:

<https://github.com/users/your0kk/packages/container/package/technomaster>

Результаты и вывод

Таблица 2 - Результаты проверки

Проверка	Результат
Линтер и TypeScript	Ошибок нет
Автоматические тесты	5 из 5
Production-сборка	Успешно
Локальная сборка Docker	Успешно
Страницы контейнера	8 страниц, HTTP 200
Healthcheck	healthy
Анонимный административный API	HTTP 401
Вход администратора в контейнере	HTTP 200
Приватные файлы в образе	Не обнаружены
Pull Request	№1, проверки пройдены, слит
Публикация GHCR	latest и полный хеш коммита
Запуск образа из GHCR	Успешно

В ходе работы приложение упаковано в Docker и настроена автоматическая проверка перед публикацией образа. Образ можно получить из GHCR и запустить с локальными настройками окружения. Размещение сайта на сервере в эту работу не входило: публикация образа в реестре подготавливает его к такому запуску.

Работа проверяет воспроизводимость сборки и основные HTTP-сценарии. Полное нагрузочное тестирование и аудит всех зависимостей не выполнялись. Предупреждения npm audit в существующих зависимостях требуют отдельного рассмотрения перед промышленным использованием.

Использованные источники

Практическое занятие №15 «Автоматизация сборки и деплоя проекта через GitHub Actions, GitLab и Docker». Методические рекомендации.

Docker. Dockerfile reference. <https://docs.docker.com/reference/dockerfile/>

Docker. Build context. <https://docs.docker.com/build/building/context/>

GitHub. Publishing Docker images.

<https://docs.github.com/en/actions/tutorials/publish-packages/publish-docker-images>

GitHub. Creating a pull request. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-a-pull-request>

Next.js. Deploying. <https://nextjs.org/docs/app/getting-started/deploying>

Исходный код проекта. <https://github.com/your0kk/technomaster>

Дата обращения к электронным источникам: 06.10.2026.